

Day05-软件包管理与文件系统

Day 05 - 软件包管理与文件系统

课程目标

- 理解 RPM 软件包结构
 - 掌握 rpm 命令查询软件包
 - 掌握 dnf 命令管理软件
 - 学会配置软件仓库
 - 理解文件系统和挂载
 - 掌握 find 和 locate 搜索文件
-

上午 - 软件包管理

第一部分：RPM 软件包基础

1.1 什么是 RPM

RPM 软件包管理器：

RPM (RPM Package Manager) 是红帽开发的软件包管理系统

- 提供标准的软件打包方式
 - 跟踪软件包安装的文件
 - 本地数据库存储已安装软件包信息
 - 红帽所有软件以 RPM 形式提供
-

RPM 软件包文件名格式：

```
name-version-release.architecture.rpm
```

示例：

```
coreutils-8.32-31.el9.x86_64.rpm
```

组成部分：

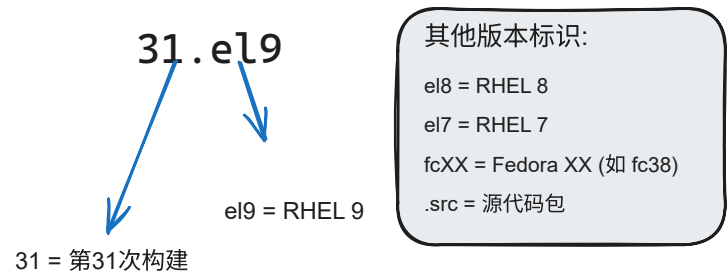
部分	说明	示例
name	软件名称	coreutils
version	原始软件版本号	8.32
release	软件包发行版本号	31.el9
architecture	处理器架构	x86_64

常见架构：

- `x86_64` - 64位Intel/AMD处理器
- `i686` / `i386` - 32位处理器
- `noarch` - 不限定架构
- `aarch64` - ARM 64位
- `ppc64le` - PowerPC 64位小端

⚠ 易错点：发行版本号含义

RPM 发行版本号说明



```
# 示例对比：
httpd-2.4.51-5.el9.x86_64.rpm # RHEL 9版本
httpd-2.4.51-1.el8.x86_64.rpm # RHEL 8版本
```

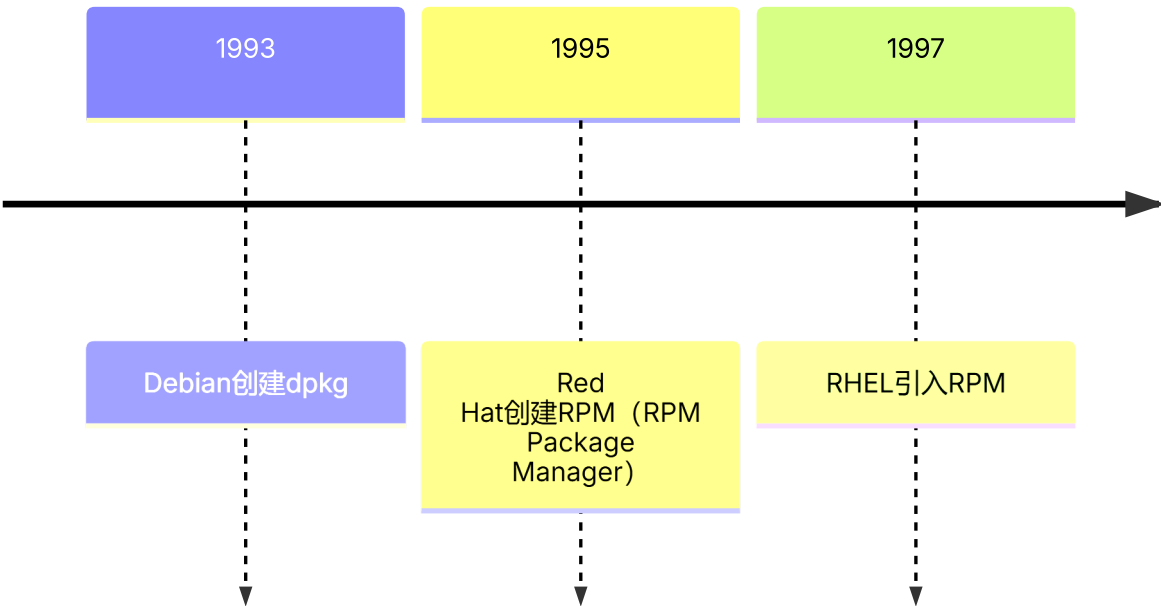
课前故事：Linux软件包管理的革命

1990年代：源代码编译时代

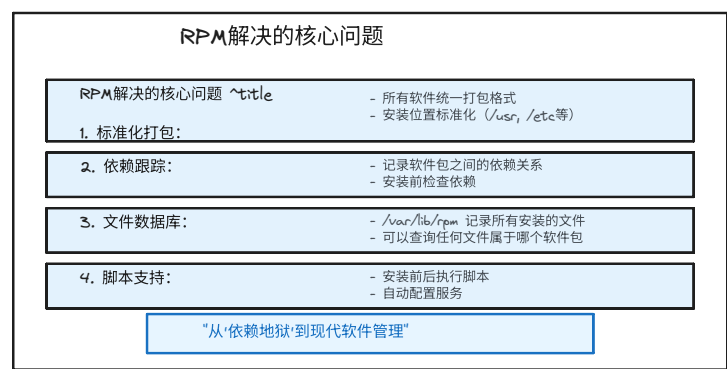
在RPM出现之前，Linux用户安装软件的方式非常原始：

![[assets/dependency-hell-flowchart: RPM的诞生**

Linux软件包管理发展史



RPM的革命性设计：



为什么RPM称为"RPM Package Manager"?

有趣的历史：

1. 最初：RPM = Red Hat Package Manager
2. 后来：RPM成为行业标准，不再专属Red Hat
3. 现在：RPM = RPM Package Manager (递归缩写)

类似例子：

- PNG = PNG Network Graphics
- GNU = GNU's Not Unix

1.2 RPM 数据库结构

RPM数据库存储位置在 `/var/lib/rpm/`，每个 **RPM** 软件包包含：

1. 安装的文件 - 实际程序文件

- 可执行文件 (/usr/bin)
- 库文件 (/usr/lib)
- 配置文件 (/etc)
- 文档文件 (/usr/share/doc)

2. 元数据 - 描述信息

- name、version、release、arch
- 软件包摘要 (Summary)
- 详细描述 (Description)
- 依赖关系 (Requires/Provides/Conflicts)
- 授权许可信息 (License)
- 更改日志 (Changelog)
- 软件包组 (Group)
- 安装大小 (Size)

3. 脚本 - 安装/更新/删除时执行的脚本

- %pre - 安装前脚本
- %post - 安装后脚本
- %preun - 卸载前脚本
- %postun - 卸载后脚本

RPM依赖关系类型：

依赖类型	说明	示例
Requires	需要的软件包	httpd requires httpd-filesystem
Provides	提供的功能	/bin/sh provides shell
Conflicts	冲突的软件包	sendmail conflicts with postfix
Obsoletes	废弃的软件包	new-package obsoletes old-package

⚠ 使用场景：查看依赖关系

SHELL

```
# 查看软件包的依赖
rpm -qR httpd

# 输出示例：
# /bin/bash
# /bin/sh
# config(httpd) = 2.4.51-5.el9
# httpd-filesystem = 2.4.51-5.el9
# httpd-tools = 2.4.51-5.el9
# libapr-1.so.0()(64bit)
# libaprutil-1.so.0()(64bit)
# ...

# 查看软件包提供的功能
rpm -qP httpd

# 输出示例：
# config(httpd) = 2.4.51-5.el9
# httpd = 2.4.51-5.el9
# httpd(x86-64) = 2.4.51-5.el9
# webserver
```

第二部分：rpm 命令查询

2.1 查询已安装软件包

rpm -q - 查询软件包：

```
# 查看已安装的软件包版本
rpm -q dnf

# 输出示例:
# dnf-4.10.0-4.el9.noarch
```

参数说明:

- `-q` = query 查询

rpm -qa - 列出所有已安装软件包:

```
# 列出所有已安装的软件包
rpm -qa

# 配合 grep 查找特定软件
rpm -qa | grep ansible

# 输出示例:
# ansible-core-2.12.2-1.el9.x86_64
```

参数说明:

- `-qa` = query all 查询所有

2.2 查询软件包详细信息

rpm -qi - 显示软件包详细信息:

```
# 显示软件包详细信息
```

```
rpm -qi dnf
```

```
# 输出示例:
```

```
# Name : dnf
```

```
# Version : 4.10.0
```

```
# Release : 4.el9
```

```
# Architecture: noarch
```

```
# Install Date: Mon 13 Jan 2025
```

```
# Group : Unspecified
```

```
# Size : 3214567
```

```
# License : GPLv2+
```

```
# Description : DNF is the package manager...
```

参数说明:

- **-i** = information 信息

2.3 查询软件包文件

rpm -ql - 列出软件包安装的所有文件:

```
# 列出软件包安装的文件
```

```
rpm -ql dnf
```

```
# 输出示例:
```

```
# /usr/bin/dnf
```

```
# /usr/lib/systemd/system/dnf-makecache.service
```

```
# /usr/share/bash-completion/completions/dnf
```

```
# ...输出省略...
```

参数说明:

- **-l** = list 列表

rpm -qc - 列出配置文件:

```
# 列出软件包的配置文件  
rpm -qc openssh-clients  
  
# 输出示例:  
# /etc/ssh/ssh_config  
# /etc/ssh/ssh_config.d/05-redhat.conf
```

参数说明:

- **-c** = config 配置文件
-

rpm -qd - 列出文档文件:

```
# 列出软件包的文档文件  
rpm -qd openssh-clients  
  
# 输出示例:  
# /usr/share/man/man1/scp.1.gz  
# /usr/share/man/man1/sftp.1.gz  
# /usr/share/man/man1/ssh.1.gz
```

参数说明:

- **-d** = doc 文档
-

2.4 查询文件所属软件包

rpm -qf - 查找文件来自哪个软件包:

```
# 查找文件属于哪个软件包  
rpm -qf /etc/hosts  
  
# 输出示例:  
# setup-2.13.6-1.el9.noarch
```

参数说明:

- `-f` = file 文件

2.5 RPM 查询命令总结

命令	作用
<code>rpm -q NAME</code>	显示软件包版本
<code>rpm -qa</code>	列出所有已安装软件包
<code>rpm -qi NAME</code>	显示软件包详细信息
<code>rpm -ql NAME</code>	列出软件包包含的所有文件
<code>rpm -qc NAME</code>	列出软件包的配置文件
<code>rpm -qd NAME</code>	列出软件包的文档文件
<code>rpm -qf FILE</code>	查找文件所属的软件包

2.6 RPM 安装、升级和卸载

`rpm -ivh` - 安装软件包:

```
# 安装 RPM 软件包
rpm -ivh package.rpm

# 参数说明:
# -i = install 安装
# -v = verbose 显示详细信息
# -h = hash marks 显示进度条

# 安装示例
rpm -ivh vsftpd-3.0.3-49.el9.x86_64.rpm

# 预期输出:
# Preparing...
##### [100%]
# Updating / installing...
# 1:vsftpd-3.0.3-49.el9.x86_64
##### [100%]
```

SHELL

依赖处理:

```
# 如果有依赖问题，需要先安装依赖
# RPM 不会自动解决依赖，需要手动安装

# 示例：安装 httpd 时需要 apr
rpm -ivh apr-1.7.0-11.el9.x86_64.rpm
rpm -ivh apr-util-1.6.1-20.el9.x86_64.rpm
rpm -ivh httpd-2.4.51-5.el9.x86_64.rpm
```

rpm -Uvh - 升级软件包：

```
# 升级软件包（或安装）
rpm -Uvh package-2.0-1.el9.x86_64.rpm

# 示例：
rpm -Uvh httpd-2.4.52-1.el9.x86_64.rpm

# 预期输出：
# Preparing...
##### [100%]
# Updating / installing...
#   1:httpd-2.4.52-1.el9.x86_64
##### [100%]
# Cleaning up / removing...
#   1:httpd-2.4.51-5.el9.x86_64
##### [100%]
```

说明：

- **-U** = upgrade 升级
- 如果软件包未安装，则会安装
- 如果旧版本存在，则会升级到新版本

rpm -e - 卸载软件包：

卸载软件包

```
rpm -e package-name
```

示例:

```
rpm -e vsftpd
```

注意: 只需软件包名称, 不需要完整版本号

警告:

- RPM 不会检查依赖关系
- 卸载可能影响依赖此软件包的其他程序
- 卸载前应确认没有其他程序依赖它

rpm -Fvh - 重新安装软件包:

重新安装软件包 (修复损坏的文件)

```
rpm -Fvh package
```

或使用 --replacepkgs

```
rpm -ivh --replacepkgs package.rpm
```

进阶技巧：RPM 验证和检查

SHELL

```
# 验证软件包完整性
rpm -V httpd

# 输出说明：
# S.5....T.  c /etc/httpd/conf/httpd.conf
# | | | | | | |
# | | | | | | +--> 文件类型
# | | | | | | +-----> 时间戳修改
# | | | | | +-----> MD5 校验
# | | | +-----> 文件大小
# | | +-----> 设备号
# | +-----> 符号链接
# +-----> 权限

# 如果没有输出，表示文件未修改
# 有输出表示文件已被修改

# 验证所有已安装软件包
rpm -Va

# 导出软件包列表（用于备份）
rpm -qa > /root/rpm-list.txt

# 从列表恢复软件包
dnf install $(cat /root/rpm-list.txt)
```

RPM 操作命令总结：

操作	命令	说明
安装	rpm -ivh package.rpm	安装软件包
升级	rpm -Uvh package.rpm	升级软件包
卸载	rpm -e package	卸载软件包
查询	rpm -q package	查询是否安装
	rpm -qa grep package	查询过滤

演示：RPM 实践

演示目标

掌握 rpm 命令查询软件包信息

演示环境要求

- 标准 RHEL 9 系统
- 无需额外权限（普通用户即可执行查询操作）
- 所有命令均为只读操作，安全无风险

Classroom 环境注意事项：

- 设备名使用 `vda` (virtio) 而非 `sda` (SATA/SCSI)
 - 根分区为 `/dev/vda4` 而非 `vda2` 或 `vda3`
 - 软件包版本可能与标准 RHEL 9 不同
 - `gpgcheck=0` (禁用 GPG 检查)
 - `blkid` 命令需要 root 权限，普通用户无输出
 - `find` 命令会产生权限拒绝警告（正常，添加 `2>/dev/null` 抑制）
-

演示步骤

步骤1：查询软件包版本

```
# 查询 bash 软件包版本
rpm -q bash

# 预期输出示例（具体版本可能略有差异）：
# bash-5.1.8-4.el9.x86_64

# 查询多个软件包
rpm -q bash coreutils systemd

# 预期输出（实际 classroom 环境版本）：
# bash-5.1.8-4.el9.x86_64
# coreutils-8.32-31.el9.x86_64
# systemd-250-6.el9_0.x86_64
```

说明：

- `-q` = query 查询模式
- 如果软件包未安装，会显示 "package xxx is not installed"

步骤2：查看软件包详细信息

查看 bash 详细信息

```
rpm -qi bash
```

预期完整输出示例:

```
# Name           : bash
# Version        : 5.1.8
# Release        : 4.el9 (classroom 环境)
# Architecture   : x86_64
# Install Date   : 2024年01月15日 星期一 10时30分00秒
# Group          : Unspecified
# Size           : 12475360
# License        : GPLv3+
# Signature      : RSA/SHA256, 2024年01月10日, Red Hat, Inc.
# Source RPM     : bash-5.1.8-4.el9.src.rpm (classroom 环境)
# Build Date     : 2024年01月08日 星期一 15时20分00秒
# Build Host     : x86-01-04.build.eng.bos.redhat.com
# Relocations    : (not relocatable)
# Packager       : Red Hat, Inc.
<http://bugzilla.redhat.com/bugzilla>
# Vendor         : Red Hat, Inc.
# URL            : https://www.gnu.org/software/bash/
# Summary        : The GNU Bourne Again shell
# Description    :
# Bash is the shell, or command language interpreter, that will
# appear in the GNU operating system. Bash is an sh-compatible
# shell that incorporates useful features from the Korn shell
# (ksh)
# and C shell (csh).
```

重点信息说明:

- **Install Date** - 安装日期
- **Size** - 安装后占用空间 (字节)
- **License** - 软件许可证 (GPLv3+ = GPL version 3 or later)

步骤3: 列出软件包文件

```
# 列出 bash 安装的所有文件
rpm -ql bash | head -20

# 预期输出示例（前20行）：
# /usr/bin/bash
# /usr/bin/sh
# /usr/share/licenses/bash
# /usr/share/licenses/bash/COPYING
# /usr/share/man/man1/bash.1.gz
# /usr/share/man/man1/bashbug.1.gz
# ...

# 查看主程序位置
which bash
# 预期输出：/usr/bin/bash

# 验证 sh 是 bash 的符号链接
ls -l /usr/bin/sh
# 预期输出：... /usr/bin/sh -> /usr/bin/bash
```

说明：

- `-l` = list 列出文件
- 输出可能很长，建议用 `head` 或 `less` 查看
- `/usr/bin/sh` 是 `/usr/bin/bash` 的软链接

步骤4：列出配置文件

```
# 列出 openssh-server 的配置文件
rpm -qc openssh-server

# 预期输出示例：
# /etc/ssh/sshd_config
# /etc/sysconfig/ssh

# 查看配置文件内容（如果文件存在）
cat /etc/ssh/sshd_config | head -10
```

说明：

- `-c` = config 配置文件
 - 配置文件通常位于 `/etc` 目录
-

步骤5：查找文件所属软件包

SHELL

```
# 查找 /etc/passwd 属于哪个软件包
rpm -qf /etc/passwd

# 预期输出: setup-2.13.7-6.el9.noarch (classroom 环境版本)

# 查找命令所属软件包
rpm -qf /usr/bin/ls
# 预期输出: coreutils-8.32-31.el9.x86_64 (classroom 环境版本)

# 查找目录所属软件包
rpm -qf /etc/ssh
# 预期输出: openssh-server-8.7p1-8.el9.x86_64
```

说明：

- `-f` = file 文件
 - 适用于文件、目录、命令等各类文件系统对象
 - 如果文件不属于任何软件包，会显示错误
-

步骤6：列出文档文件

SHELL

```
# 列出软件包的文档文件
rpm -qd openssh-clients

# 预期输出示例：
# /usr/share/man/man1/sftp.1.gz
# /usr/share/man/man1/ssh.1.gz
# /usr/share/man/man1/scp.1.gz
# /usr/share/doc/openssh-clients/README.dns
```

说明：

- `-d` = doc 文档

- 文档通常包括 man 手册页和 `/usr/share/doc` 下的文件

演示总结

命令	功能	典型用途
<code>rpm -q 包名</code>	查看软件包版本	检查软件是否安装
<code>rpm -qi 包名</code>	查看详细信息	了解软件来源、大小、许可证
<code>rpm -ql 包名</code>	列出所有文件	找到软件安装位置
<code>rpm -qc 包名</code>	列出配置文件	编辑配置前先备份
<code>rpm -qd 包名</code>	列出文档文件	查看帮助文档
<code>rpm -qf 文件</code>	查找文件所属包	误删文件后恢复

步骤7：RPM 安装/卸载操作（演示）

 注意：以下操作需要 root 权限，且会修改系统状态
建议在测试环境演示，或仅展示命令格式

```
# 查看当前是否安装 tree
# 如果已经安装了, 可以先执行卸载 sudo dnf remove -y tree
rpm -q tree
# 预期: package tree is not installed

# 演示: 安装 tree (使用 dnf 代替 rpm -ivh, 更安全)
sudo dnf install -y tree
# 预期输出: Complete!

# 验证安装
rpm -q tree
# 预期: tree-1.x.x-xxx.el9.x86_64

# 查看安装的文件
rpm -ql tree | head -10
# 预期: /usr/bin/tree, /usr/share/man/...

# 卸载演示
sudo dnf remove -y tree
# 预期输出: Complete!

# 验证卸载
rpm -q tree
# 预期: package tree is not installed
```

说明:

- 工作中优先建议使用 `dnf` 而非直接 `rpm -ivh`
- `dnf` 会自动处理依赖关系
- 使用 `tree` 作为演示软件包, 因为它小巧无依赖

故障排除

问题1: 软件包未安装

```
# 症状:
rpm -q nginx
# 输出: package nginx is not installed

# 解决: 使用 dnf 安装
dnf install nginx
```

问题2：文件不属于任何软件包

```
# 症状:
rpm -qf /root/myfile.txt
# 输出: error: file /root/myfile.txt is not owned by any package

# 说明: 这是正常的，用户自己创建的文件不属于任何软件包
```

相关练习：

练习名称	对应章节	说明
练习：EXPLAINING AND INVESTIGATING RPM SOFTWARE PACKAGES	第12章 第02节	查询和调查 RPM软件包

第三部分：DNF 软件包管理

课前故事：从 RPM 到 DNF 的演进

RPM 的局限性：

虽然 RPM 解决了软件打包的问题，但它有一个重要限制：

RPM 的问题：无法自动解决依赖关系

场景：安装 httpd Web服务器

```
rpm -ivh httpd-2.4.51-5.el9.x86_64.rpm
```

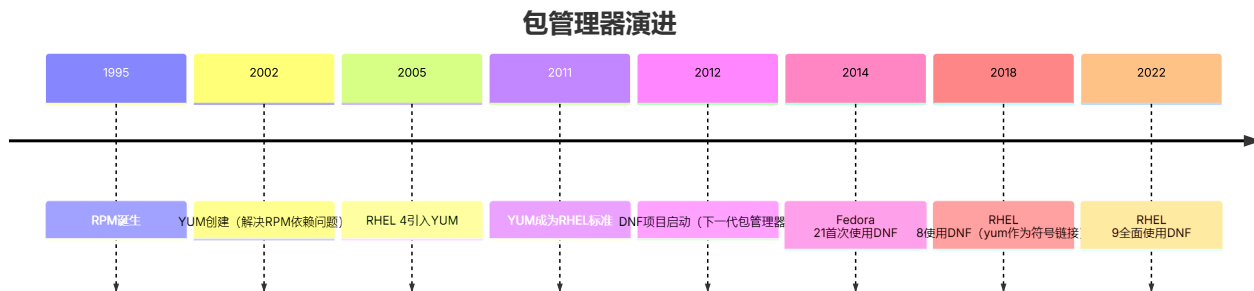
错误输出：

```
error: Failed dependencies:
    apr >= 1.7.0 is needed by httpd-2.4.51-5.el9.x86_64
    apr-util >= 1.6.1 is needed by httpd-2.4.51-5.el9.x86_64
    httpd-tools is needed by httpd-2.4.51-5.el9.x86_64
    ...
```

管理员需要：

1. 手动找到 apr 的 RPM 包
2. 安装 apr
3. 发现 apr 又需要其他依赖
4. 重复上述过程 → "依赖地狱"再次出现！

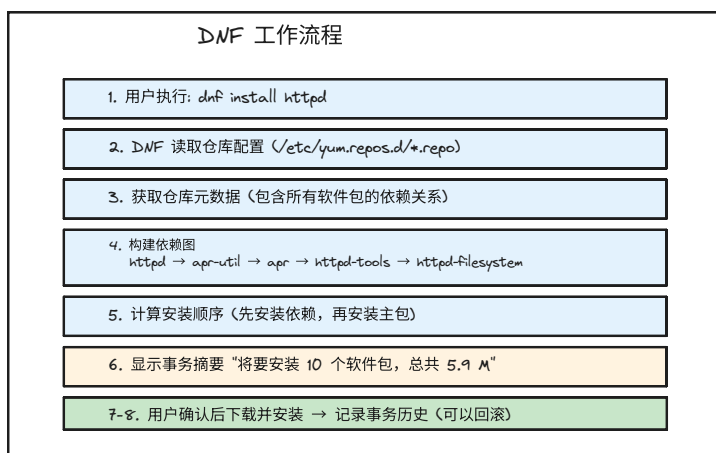
YUM 的诞生（2000年代）：



DNF 为什么取代 YUM？

DNF通过性能革命、功能增强和生态适配，解决了YUM时代的核心痛点，成为现代Linux发行版的必然选择。对于不联网的私有环境，DNF的本地源依赖解析优势更明显。

DNF 工作原理：



3.1 DNF 介绍

什么是 **DNF**:

DNF (Dandified YUM) 是 RHEL9 的默认软件包管理器

- 取代了 YUM, 但 `yum` 命令作为符号链接仍然可用
- 自动解决依赖关系
- 支持软件仓库操作
- 提供事务历史记录
- 基于 `libsolv` 库进行依赖解析

yum 和 **dnf** 的关系:

SHELL

```
# yum 是 dnf 的符号链接
ls -l /usr/bin/yum

# 输出示例:
# ... /usr/bin/yum -> dnf-3

# 验证:
file /usr/bin/yum
# 输出: /usr/bin/yum: symbolic link to /usr/bin/dnf-3
```

⚠ 使用场景: **yum vs dnf**

```
# 在 RHEL9 中:
yum install httpd      # 实际调用的是 dnf
dnf install httpd      # 直接调用 dnf

# 两者完全等效, 但推荐使用 dnf

# 查看实际程序:
which dnf
# /usr/bin/dnf

rpm -qf /usr/bin/dnf
# dnf-4.10.0-4.el9.noarch
```

3.2 DNF 基本操作

dnf list - 列出软件包:

```
# 列出所有软件包
dnf list

# 列出已安装的软件包
dnf list installed

# 列出可用的软件包
dnf list available

# 搜索特定软件包
dnf list 'http*'
```

输出说明:

- **Installed Packages** - 已安装的软件包
- **Available Packages** - 可安装的软件包

dnf search - 搜索软件包:

```
# 搜索软件包（在名称、摘要、描述中查找）
dnf search web

# 输出示例：
# Name and Summary Matched:
# nginx.x86_64 : A high performance web server
# httpd.x86_64 : Apache HTTP Server
```

dnf info - 查看软件包详细信息：

```
# 查看软件包详细信息
dnf info httpd

# 输出示例：
# Available Packages
# Name           : httpd
# Version        : 2.4.51
# Release        : 5.el9
# Architecture   : x86_64
# Size           : 1.5 M
# Source         : httpd-2.4.51-5.el9.src.rpm
# Repository     : rhel-9-for-x86_64-appstream-rpms
# Summary        : Apache HTTP Server
# URL            : https://httpd.apache.org/
# License        : ASL 2.0
# Description    : The Apache HTTP Server...
```

3.3 安装和删除软件包

dnf install - 安装软件包：


```
# 安装软件包
dnf install httpd

# 安装过程:
# Dependencies resolved:
# Installing:
# httpd.x86_64 2.4.51-5.el9
# Installing dependencies:
# apr.x86_64 1.7.0-11.el9
# apr-util.x86_64 1.6.1-20.el9
# ...
# Transaction Summary:
# Install 10 Packages
# Total download size: 2.1 M
# Installed size: 5.9 M
# Is this ok [y/N]: y
```

说明:

- dnf 自动解决依赖关系
- 显示要安装的软件包和依赖项
- 需要确认安装

dnf remove - 删除软件包:

```
# 删除软件包
dnf remove httpd

# 删除过程:
# Dependencies resolved:
# Removing:
# httpd.x86_64 2.4.51-5.el9
# Removing dependencies:
# apr-util-openssl.x86_64 1.6.1-20.el9
# ...
# Transaction Summary:
# Remove 10 Packages
# Freed space: 5.9 M
# Is this ok [y/N]: y
```

dnf update - 更新软件包:

SHELL

```
# 更新所有软件包
dnf update

# 更新特定软件包
dnf update httpd

# 检查可用更新
dnf check-update
```

3.4 DNF 历史记录

dnf history - 查看事务历史:

SHELL

```
# 查看 DNF 事务历史
dnf history

# 输出示例:
# ID      | Command                      | Date and time      | Action(s)
# | Altered
# -----+-----+-----+-----
# 9 | install httpd                | 2025-01-15 10:00 | Install
# 10
# 8 | group install RPM            | 2025-01-15 09:00 | Install
# 20
# 7 | update                       | 2025-01-14 16:00 | Update
# 31
```

输出说明:

- ID - 事务编号
- Command - 执行的命令
- Action(s) - 操作类型
- Altered - 变更的软件包数量

dnf history undo - 撤销事务:

```
# 撤销指定事务
dnf history undo 9

# 说明:
# 撤销事务ID为9的操作
# 如果是安装, 则会删除
# 如果是删除, 则会重新安装
```

进阶技巧: dnf 模块流

⚠ RHEL9 新特性: 模块流 (Module Streams)

```
# 什么是模块流?
# - 允许同一软件的多个版本共存
# - 例如: Python 3.8、Python 3.9、PostgreSQL 10、12、14

# 查看可用模块
dnf module list

# 输出示例:
# Name                Stream          Profiles
# postgresql          13 [d]          client, server [d]
# postgresql           15              client, server
# python38             2.7 [d]         common [d]
# python39             3.9             common [d]

# [d] = 默认流

# 查看特定模块详情
dnf module info postgresql

# 启用特定模块流
dnf module enable postgresql:15

# 安装模块化软件包
dnf install postgresql-server

# 重置模块选择
dnf module reset postgresql
```

⚠ 使用场景: 模块流

```
# 实际工作中可能要求：
# 1. 启用非默认版本的软件
# 2. 安装特定版本的软件包

# 示例：安装 PostgreSQL 15
dnf module enable postgresql:15
dnf install postgresql-server

# 验证安装的版本
dnf info postgresql-server
```

dnf 模块流 不在RHCSA考纲中, 了解即可

练习名称	对应章节	说明
练习：INSTALLING AND UPDATING SOFTWARE PACKAGES WITH DNF	第12章 第03节	使用DNF安装和更新软件包

第四部分：软件仓库管理

4.1 查看软件仓库

dnf repolist - 列出软件仓库：

```
# 列出启用的软件仓库
```

```
dnf repolist
```

```
# 输出示例:
```

# repo id	repo name
# appstream	RHEL 9 AppStream
# baseos	RHEL 9 BaseOS

```
# 列出所有软件仓库
```

```
dnf repolist all
```

```
# 输出示例:
```

# repo id	status	repo name
# appstream	enabled	RHEL 9 AppStream
# baseos	enabled	RHEL 9 BaseOS
# debug-appstream	disabled	RHEL 9 AppStream Debug

参数说明:

- **all** - 显示所有仓库（包括禁用的）

4.2 配置软件仓库

仓库配置文件位置:

```
/etc/yum.repos.d/*.repo
```

仓库配置文件格式:

```
# /etc/yum.repos.d/local.repo
[BaseOS]
name=BaseOS Repository
baseurl=http://content/rhel9.0/x86_64/dvd/BaseOS
enabled=1
gpgcheck=0

[AppStream]
name=AppStream Repository
baseurl=http://content/rhel9.0/x86_64/dvd/AppStream
enabled=1
gpgcheck=0
```

配置项说明：

- **[name]** - 仓库ID（方括号内）
- **name** - 仓库名称
- **baseurl** - 仓库地址
- **enabled** - 是否启用（1=启用，0=禁用）
- **gpgcheck** - 是否检查GPG签名

dnf config-manager - 管理仓库：

```
# 启用仓库
dnf config-manager --enable repo-id

# 禁用仓库
dnf config-manager --disable repo-id

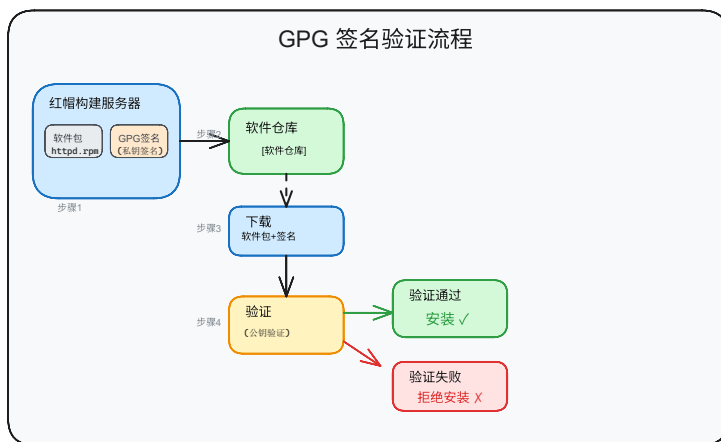
# 添加仓库
dnf config-manager --add-repo http://example.com/repo
```

4.3 GPG 签名验证

什么是 GPG 签名？

GPG (GNU Privacy Guard) 签名是一种数字签名机制，用于验证软件包的：

- 完整性：软件包未被篡改
- 真实性：软件包确实来自声称的发布者



GPG 相关配置项：

在 `/etc/yum.repos.d/*.repo` 文件中：

```
[baseos]
name=BaseOS Repository
baseurl=http://content/rhel9.0/x86_64/dvd/BaseOS
enabled=1
gpgcheck=1                # 启用 GPG 检查
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release # GPG
公钥位置
```

配置项说明：

- `gpgcheck=1` - 启用 GPG 签名验证（默认值）
- `gpgcheck=0` - 禁用 GPG 签名验证（仅用于可信的本地仓库）
- `gpgkey` - 指定 GPG 公钥文件位置

导入 GPG 公钥：

```
# 方法1: 使用 rpm 导入公钥
rpm --import /etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

# 方法2: 从 URL 导入
rpm --import https://www.redhat.com/security/data/fd431d51.txt

# 验证已导入的公钥
rpm -q gpg-pubkey

# 预期输出示例:
# gpg-pubkey-fd431d51-6ae6b46b

# 查看公钥详细信息
rpm -qi gpg-pubkey-fd431d51-6ae6b46b

# 预期输出示例:
# Name          : gpg-pubkey
# Version       : fd431d51
# Release       : 6ae6b46b
# Architecture: noarch
# Install Date: (安装日期)
# Group         : Public Keys
# Size          : 0
# License       : pubkey
# Signature     : (none)
# Source RPM    : (none)
# Build Date    : (创建日期)
# Summary       : gpg(Red Hat, Inc. (release key 2)
<security@redhat.com>)
# Description :
# -----BEGIN PGP PUBLIC KEY BLOCK-----
# ...
```

验证软件包签名:


```
# 使用 rpm 验证单个软件包
```

```
rpm -K /path/to/package.rpm
```

```
# 预期输出示例（签名有效）：
```

```
# /path/to/package.rpm: RSA/SHA256 Signature, key ID  
199e2f91fd431d51: OK
```

```
#
```

↑

↑

```
#
```

签名算法

公钥ID（红帽官

```
方）
```

```
# 预期输出示例（签名无效）：
```

```
# /path/to/package.rpm: RSA/SHA256 Signature, key ID  
199e2f91fd431d51: BAD
```

```
#
```

↑

```
#
```

```
签名验证失败
```

```
# 详细验证模式
```

```
rpm -Kv /path/to/package.rpm
```

```
# 预期输出示例：
```

```
# Header V3 RSA/SHA256 Signature, key ID 199e2f91fd431d51: OK
```

```
# Header SHA256 digest: OK
```

```
# V3 RSA/SHA256 Signature, key ID 199e2f91fd431d51: OK
```

```
# SHA256 digest: OK
```

```
# MD5 digest: OK
```

GPG 验证失败处理：

```
# 症状: 安装时 GPG 验证失败
dnf install httpd
# ...
# Public key for httpd-2.4.53-7.el9.x86_64.rpm is not installed
# Failing package is: httpd-2.4.53-7.el9.x86_64
# GPG Keys are configured as:
https://access.redhat.com/security/data/fd431d51.txt

# 原因: 系统未导入红帽 GPG 公钥

# 解决方案1: 导入公钥后重试
rpm --import /etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
dnf install httpd

# 解决方案2: 临时禁用 GPG 检查 (不推荐, 仅用于测试)
dnf install httpd --nogpgcheck

# 解决方案3: 修改仓库配置 (仅用于可信的本地仓库)
# 编辑 /etc/yum.repos.d/local.repo
# 将 gpgcheck=1 改为 gpgcheck=0
```

GPG 签名验证场景对比:

场景	gpgcheck 设置	说明
红帽官方在线仓库	<code>gpgcheck=1</code>	必须启用, 确保软件包来源可靠
红帽官方 DVD/ISO 仓库	<code>gpgcheck=1</code>	建议启用, 需先导入公钥
内网可信仓库	<code>gpgcheck=0</code>	可禁用, 因为仓库由自己控制
第三方仓库	<code>gpgcheck=1</code>	必须启用, 并验证第三方公钥

⚠ 注意:

- 1. 使用配置好的内部可信仓库, `gpgcheck` 可设为 0
- 2. 如果遇到 GPG 错误, 使用 `--nogpgcheck` 选项可以绕过

演示：DNF 实践

演示目标

掌握 dnf 命令管理软件包

演示环境要求

- RHEL 9 系统
 - 需要 root 权限 (sudo 或直接 root)
 - 配置好的软件仓库
 - 网络连接 (如果使用在线仓库)
-

演示步骤

步骤1：查看软件仓库状态

列出启用的软件仓库

```
dnf repolist
```

预期输出示例：

repo id	repo name
appstream	RHEL 9 AppStream
baseos	RHEL 9 BaseOS

查看仓库详细信息

```
dnf repolist --verbose
```

检查可用更新

```
dnf check-update | head -10
```

SHELL

说明：

- 确认仓库配置正确
 - 如果没有输出，说明没有可用更新
-

步骤2：搜索软件包

SHELL

```
# 搜索 web 服务器软件
```

```
dnf search web server
```

```
# 预期输出示例：
```

```
# Name and Summary Matched: httpd.x86_64 : Apache HTTP Server
```

```
# Name and Summary Matched: nginx.x86_64 : A high performance web  
server
```

```
# 搜索 FTP 相关软件
```

```
dnf search ftp
```

```
# 预期输出示例：
```

```
# vsftpd.x86_64 : Very Secure Ftp Daemon
```

```
# ftp.x86_64 : The client for the File Transfer Protocol (FTP)
```

```
# 按名称搜索（精确匹配）
```

```
dnf list vsftpd
```

```
# 预期输出示例：
```

```
# Available Packages
```

```
# vsftpd.x86_64 3.0.5-5.el9 baseos
```

说明：

- **search** 在名称、摘要、描述中查找
- **list** 按名称精确匹配

步骤3：查看软件包详细信息

```
# 查看 vsftpd 软件包信息
```

```
dnf info vsftpd
```

```
# 预期完整输出示例:
```

```
# Available Packages
```

```
# Name : vsftpd
```

```
# Version : 3.0.5
```

```
# Release : 5.el9
```

```
# Architecture : x86_64
```

```
# Size : 355 k
```

```
# Source : vsftpd-3.0.3-49.el9.src.rpm
```

```
# Repository : baseos
```

```
# Summary : Very Secure Ftp Daemon
```

```
# URL : https://security.appspot.com/vsftpd.html
```

```
# License : GPLv2 with exceptions
```

```
# Description : vsftpd is a Very Secure FTP daemon. It was  
written
```

```
# : completely from scratch to address security
```

```
# : concerns and performance issues...
```

```
# 查看已安装的软件包
```

```
dnf list installed | grep bash
```

```
# 预期输出:
```

```
# bash.x86_64 5.1.8-9.el9 @anaconda
```

```
# 注: @anaconda 表示系统安装时自带
```

说明:

- **Repository** 显示软件包来源
- **@anaconda** 表示系统安装时包含

步骤4: 安装软件包 (演示)

查看当前安装历史 (安装前)

```
dnf history | head -10
```

预期输出示例:

```
# ID      | Command                | Date and time      | Action(s)  |
Altered
# -----+-----+-----+-----+-----+
#      8 | update                | 2025-01-14 16:00 | E, I, U    |
31
#      7 | install vim           | 2025-01-14 10:00 | I          |
1
#      6 | install tree          | 2025-01-13 15:00 | I          |
1
```

安装一个轻量级工具进行演示 (tree 或 sl)

注意: 实际演示时可以选择其他小工具

```
dnf install -y tree
```

预期输出示例:

```
# Dependencies resolved.
# Installing:
# tree.x86_64 0:1.8.0-7.el9
#
# Complete!
```

验证安装

```
rpm -q tree
```

预期输出: tree-1.8.0-7.el9.x86_64

查看新的安装历史

```
dnf history | head -5
```

预期输出 (注意新增的事务):

```
# ID      | Command                | Date and time      | Action(s)  |
Altered
# -----+-----+-----+-----+-----+
#      9 | install tree          | 2025-01-15 10:30 | I          |
1 <-- 新增
#      8 | update                | 2025-01-14 16:00 | E, I, U    |
31
```

参数说明:

- `-y` = 自动确认安装（不需要手动输入 y）

步骤5：使用 `history undo` 撤销操作

SHELL

```
# 查看最新的事务详情
dnf history info 9

# 预期输出示例：
# Transaction ID : 9
# Begin time      : 2025-01-15 10:30:00
# Command line    : install -y tree
# Packages installed:
#     tree-1.8.0-7.el9.x86_64

# 撤销刚才的安装（注意：使用实际的事务ID）
dnf history undo 9

# 预期输出：
# Operation aborted.
# 或
# Removed:
#     tree-1.8.0-7.el9.x86_64
# Complete!

# 验证软件包已被删除
rpm -q tree
# 预期输出: package tree is not installed

# 查看 history redo（重做操作）
dnf history redo 9

# 预期输出：
# Installing:
#     tree-1.8.0-7.el9.x86_64
# Complete!
```

重要说明：

- ⚠ 使用 `history undo` 时必须使用实际的事务ID
- 事务ID 每次操作都会递增
- `undo` = 撤销（安装变删除，删除变安装）

- **redo** = 重做（重新执行该事务）

步骤6：查看事务详细信息

SHELL

```
# 查看所有历史记录
dnf history list all

# 查看特定事务的详细信息
dnf history info 8

# 预期输出包含：
# - Transaction ID: 事务编号
# - Begin time: 开始时间
# - Command line: 执行的命令
# - User: 执行用户
# - Return code: 返回码
# - Packages installed/removed/erased: 软件包变更列表

# 只查看最近5条事务
dnf history list | head -10
```

演示总结

操作	命令	说明
搜索软件	<code>dnf search keyword</code>	在名称、摘要、描述中搜索
列出软件	<code>dnf list package</code>	精确匹配软件包名称
查看信息	<code>dnf info package</code>	显示软件包详细信息
安装软件	<code>dnf install package</code>	安装软件包
删除软件	<code>dnf remove package</code>	删除软件包
查看历史	<code>dnf history</code>	列出所有事务
撤销事务	<code>dnf history undo ID</code>	撤销指定ID的事务
重做事务	<code>dnf history redo ID</code>	重做指定ID的事务

故障排除

问题1: 仓库无法访问

SHELL

```
# 症状:
dnf install tree
# Error: Failed to synchronize cache for repo 'baseos'

# 排查步骤:
# 1. 检查网络连接
ping -c 3 classroom.example.com

# 2. 查看仓库配置
cat /etc/yum.repos.d/*.repo

# 3. 清除缓存重试
dnf clean all
dnf makecache

# 4. 检查 SELinux 状态 (如有问题)
getenforce
```

问题2: 软件包冲突

SHELL

```
# 症状:
dnf install httpd
# Error: Problem: problem with installed package

# 解决方案:
# 查看冲突详情
dnf install httpd --skip-broken

# 或使用 --alloweraseing 允许替换冲突包
dnf install httpd --alloweraseing
```

问题3: history undo 失败

```
# 症状:
dnf history undo 9
# Error: Transaction ID 9 not found

# 原因: 事务ID不存在或已被清理

# 解决: 查看当前有效的事务ID
dnf history

# 使用正确的ID
dnf history undo 8
```

练习名称	对应章节	说明
练习：ENABLING DNF SOFTWARE REPOSITORIES	第12章 第04节	启用DNF软件仓库
总练习：INSTALLING AND UPDATING SOFTWARE PACKAGES	第12章	软件包管理综合练习

下午 - 文件系统与文件搜索

课前故事：Linux文件系统的演进

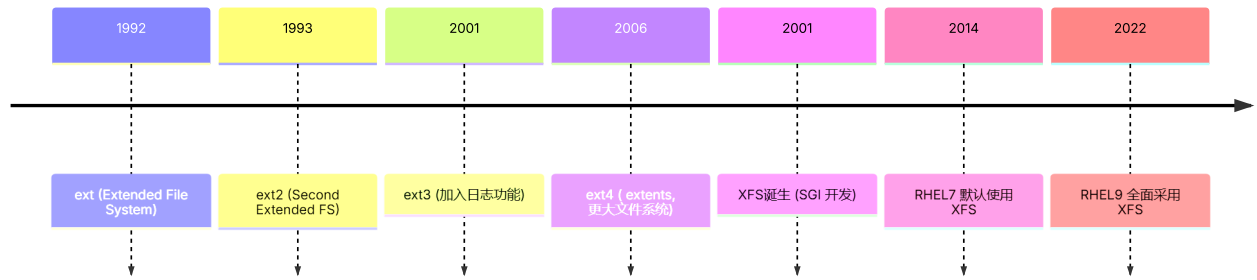
什么是文件系统？

文件系统是操作系统用于管理存储设备上数据的方法和结构。它决定了：

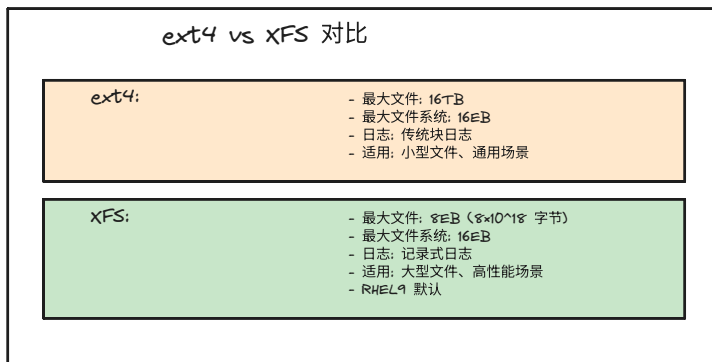
- 文件如何存储和检索
- 目录如何组织
- 空间如何分配
- 元数据如何管理

Linux 文件系统发展史：

Linux文件系统演进



ext4 vs XFS 对比:



为什么 RHEL 选择 XFS?

技术原因:

1. 高性能:
 - B+树索引结构
 - 延迟分配 (delayed allocation)
 - 大文件性能优异
2. 高可靠性:
 - 元数据日志
 - 在线检查和修复 (xfs_repair)
 - 快速恢复
3. 可扩展性:
 - 支持超大文件 (8EB)
 - 支持超大文件系统 (16EB)
 - 动态空间分配 (不需预分配 inode)
4. 企业特性:
 - 配额管理 (project quota)
 - 快照支持
 - 数据段管理

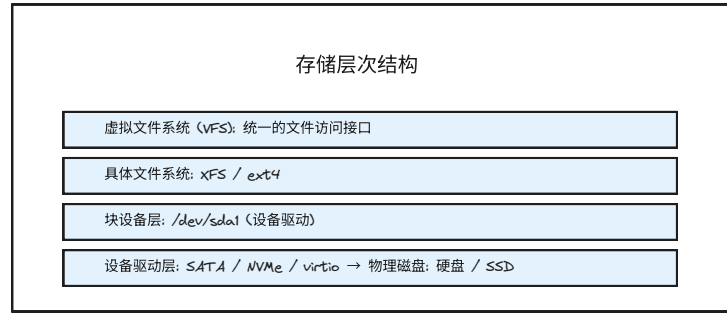
第五部分：文件系统基础

5.1 文件系统概述

Linux 支持的文件系统：

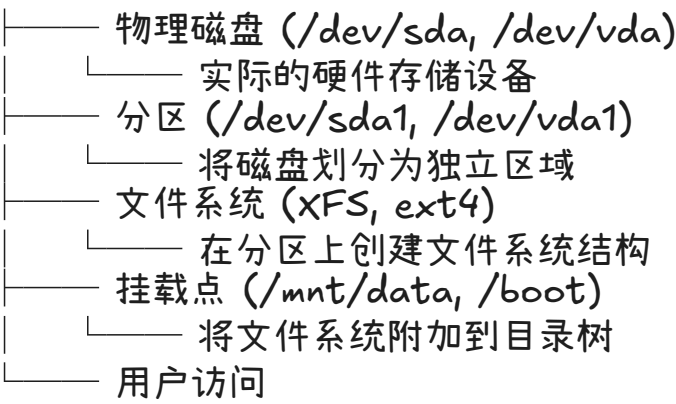
文件系统	说明	用途	重要性
XFS	RHEL9 默认文件系统	本地文件系统	☆☆☆
ext4	传统文件系统	本地文件系统	☆☆
exFAT	可移动介质	U盘、SD卡	☆
NFS	网络文件系统	网络共享	☆
GFS2	集群文件系统	集群共享	☆
Btrfs	写时复制文件系统	快照、压缩	☆

存储层次结构：



存储层次详解：

存储层次：



⚠ 必须掌握的概念：挂载（Mount）

SHELL

```
# 挂载是将文件系统连接到目录树的过程

# 没有挂载的文件系统无法访问
ls /mnt/data
# 空目录或不存在

# 挂载后才能访问
mount /dev/vdb1 /mnt/data
ls /mnt/data
# 显示文件系统文件

# 挂载点就是一个目录
# 任何空目录都可以作为挂载点
```

5.2 块设备命名

块设备命名规则：

设备类型	命名模式	示例
SATA/SAS/USB	/dev/sda, /dev/sdb	/dev/sda1
virtio-blk	/dev/vda, /dev/vdb	/dev/vda1
NVMe SSD	/dev/nvme0, /dev/nvme1	/dev/nvme0n1p1
SD/MMC	/dev/mmcblk0	/dev/mmcblk0p1

分区命名：

传统磁盘：

/dev/sda1 - 第一磁盘第一分区

/dev/sda2 - 第一磁盘第二分区

/dev/sdb3 - 第二磁盘第三分区

NVMe 磁盘：

/dev/nvme0n1p1 - 第一NVMe磁盘第一分区

/dev/nvme1n1p2 - 第二NVMe磁盘第二分区

5.3 查看文件系统

lsblk - 列出块设备：

列出所有块设备

lsblk

输出示例：

#	NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
#	vda	253:0	0	40G	0	disk	
#	└vda1	253:1	0	200M	0	part	/boot/efi
#	└vda2	253:2	0	500M	0	part	/boot
#	└┬vda3	253:3	0	39.3G	0	part	/

输出说明：

- NAME - 设备名称
- MAJ:MIN - 主次设备号
- RM - 可移动(1=可移动, 0=不可移动)
- SIZE - 设备大小
- RO - 只读
- TYPE - 类型(disk磁盘, part分区)
- MOUNTPOINT - 挂载点

lsblk -f - 查看文件系统类型和UUID：

查看文件系统信息

lsblk -f

输出示例:

NAME FSTYPE LABEL UUID

MOUNTPOINT

vda

└─vda1 vfat 7B77-95E7 /boot/efi

└─vda2 xfs 2d67e6d0-1f09-... /boot

└─vda3 xfs efd314d0-b56e-... /

输出说明:

- FSTYPE - 文件系统类型
- LABEL - 卷标
- UUID - 唯一标识符
- MOUNTPOINT - 挂载点

df - 查看磁盘使用情况:

查看文件系统磁盘使用

df

人类可读格式

df -h

输出示例:

Filesystem Size Used Avail Use% Mounted on

devtmpfs 892M 0 892M 0% /dev

/dev/vda3 8.0G 1.4G 6.7G 17% /

/dev/vda1 511M 101M 411M 20% /boot

输出说明:

- Filesystem - 文件系统
- Size - 总大小
- Used - 已使用
- Avail - 可用空间

- Use% - 使用百分比
- Mounted on - 挂载点

参数说明：

- `-h` - 人类可读格式(KiB, MiB, GiB)
- `-H` - SI单位(KB, MB, GB)

第六部分：挂载文件系统

6.1 mount - 挂载文件系统

mount 命令语法：

```
mount [设备] [挂载点]
```

SHELL

基本挂载：

```
# 创建挂载点
mkdir /mnt/data

# 挂载文件系统
mount /dev/vdb1 /mnt/data
```

SHELL

参数说明：

- 第一个参数 - 要挂载的设备
- 第二个参数 - 挂载点目录

按 **UUID** 挂载（推荐）：


```
# 使用 UUID 挂载
mount UUID="efd314d0-b56e-45db-bbb3-3f32ae98f652" /mnt/data

# 查看设备 UUID
lsblk -f
```

说明：

- UUID 稳定且唯一
- 推荐使用 UUID 而不是设备名
- 设备名可能会变化

指定文件系统类型：

```
# 指定文件系统类型
mount -t xfs /dev/vdb1 /mnt/data

# 参数说明：
# -t xfs = 指定文件系统类型为 XFS
```

6.2 umount - 卸载文件系统

umount 命令：

```
# 卸载文件系统
umount /mnt/data

# 也可以使用设备名
umount /dev/vdb1
```

设备忙时无法卸载：

```
# 如果文件系统正在使用，会报错
umount /mnt/data

# 错误输出：
# umount: /mnt/data: target is busy.

# 查看谁在使用该目录
lsof /mnt/data

# 输出示例：
# COMMAND      PID  USER   FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME
# bash          1234 root    cwd  DIR    253,17      128    /mnt/data
```

解决方案：

1. 退出使用该目录的进程
2. 使用 `lsof` 查找占用进程
3. 使用 `fuser -km /mnt/data` 强制终止进程

6.3 blkid - 查看块设备属性

blkid 命令：

blkid (block ID) 命令用于定位和显示块设备属性，特别是 UUID 和文件系统类型。

 **Classroom** 环境注意事项：

- **blkid** 命令在 classroom 环境中需要 **root** 权限才能输出结果
- 普通用户执行 **blkid** 可能无法读取某些特殊分区的完整信息

显示所有已挂载和可用的块设备

```
sudo blkid
```

预期输出示例:

```
# /dev/sda1: UUID="8b1b1c1d-1e1f-1a1a-1b1b-1c1c1d1d1e1e"
```

```
TYPE="xfs" PARTUUID="1a1b1c1d-01"
```

```
# /dev/sda2: UUID="efd314d0-b56e-45db-bbb3-3f32ae98f652"
```

```
TYPE="ext4" PARTUUID="1a1b1c1d-02"
```

```
# /dev/sr0: UUID="2023-03-15-12-34-56-00" LABEL="RHEL-9-0-0-BaseOS-x86_64" TYPE="iso9660"
```

```
#          ↑ 设备名
```

```
          ↑ UUID (唯一标识符)
```

```
          ↑
```

```
文件系统类型
```

常用选项:

只显示指定设备的属性

```
blkid /dev/sda1
```

预期输出:

```
# /dev/sda1: UUID="8b1b1c1d-1e1f-1a1a-1b1b-1c1c1d1d1e1e"  
TYPE="xfs"
```

只显示 UUID (用于脚本)

```
blkid -s UUID -o value /dev/sda1
```

预期输出:

```
# 8b1b1c1d-1e1f-1a1a-1b1b-1c1c1d1d1e1e
```

只显示文件系统类型

```
blkid -s TYPE -o value /dev/sda1
```

预期输出:

```
# xfs
```

显示所有可用标签

```
blkid -o udev
```

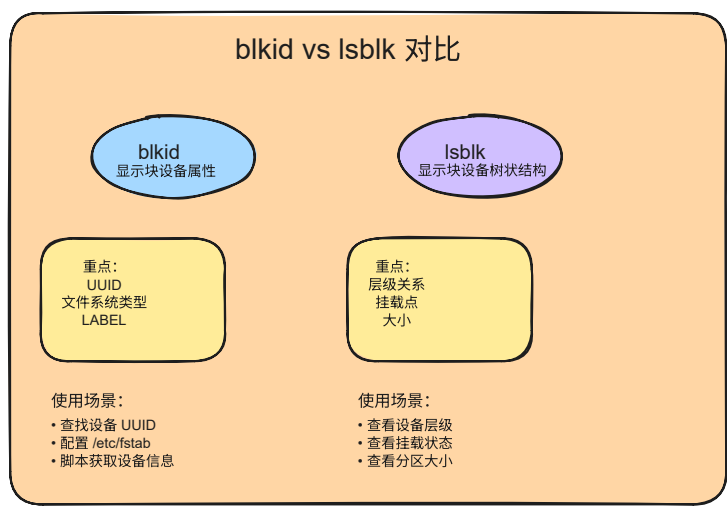
预期输出示例:

```
# ID_FS_UUID=8b1b1c1d-1e1f-1a1a-1b1b-1c1c1d1d1e1e  
# ID_FS_TYPE=xfs  
# ID_FS_LABEL=  
# ID_FS_LABEL_ENC=  
# ID_FS_USAGE=filesystem
```

参数说明:

选项	说明
<code>-s 标签</code>	只显示指定标签 (如 UUID、TYPE、LABEL)
<code>-o value</code>	只显示值, 不显示标签名
<code>-o device</code>	只显示设备名
<code>-o list</code>	以列表格式显示
<code>-o udev</code>	以 udev 格式显示 (供脚本使用)

blkid 与 lsblk 对比：



实际应用示例：

```
# 场景1: 复制文件系统到另一个设备 (保留 UUID)
# 先获取源设备的 UUID
SRC_UUID=$(blkid -s UUID -o value /dev/sda1)
echo "源设备 UUID: $SRC_UUID"

# 使用 xfs_copy 复制并验证
xfs_copy /dev/sda1 /dev/sdb1
NEW_UUID=$(blkid -s UUID -o value /dev/sdb1)
echo "新设备 UUID: $NEW_UUID"

# 场景2: 为特定设备创建挂载点
DEVICE="/dev/sdb1"
UUID=$(blkid -s UUID -o value $DEVICE)
MOUNT_POINT="/mnt/data_$(echo $UUID | cut -c1-8)"

mkdir -p "$MOUNT_POINT"
mount "$DEVICE" "$MOUNT_POINT"
echo "已挂载到: $MOUNT_POINT"

# 场景3: 查找所有 ext4 文件系统
for dev in $(lsblk -d -n -o NAME); do
    if blkid -s TYPE -o value "/dev/$dev" 2>/dev/null | grep -q
"ext4"; then
        echo "发现 ext4 设备: /dev/$dev"
        blkid "/dev/$dev"
    fi
done

# 预期输出:
# 发现 ext4 设备: /dev/sda2
# /dev/sda2: UUID="efd314d0-b56e-45db-bbb3-3f32ae98f652"
TYPE="ext4"
```

实际应用示例:

```
# 场景1: 配置持久化挂载
# 需要获取设备的 UUID

# 方法1: 使用 blkid (推荐)
blkid /dev/vdb1
# /dev/vdb1: UUID="a1b2c3d4-e5f6-4a5b-8c7d-9e0f1a2b3c4d"
TYPE="xfs"

# 方法2: 使用 lsblk -f
lsblk -f /dev/vdb1
# NAME FSTYPE LABEL UUID
MOUNTPOINT
# vdb1 xfs          a1b2c3d4-e5f6-4a5b-8c7d-9e0f1a2b3c4d

# 然后编辑 /etc/fstab
UUID=a1b2c3d4-e5f6-4a5b-8c7d-9e0f1a2b3c4d /mnt/data xfs
defaults 0 0
```

故障排除:

```
# 问题1: 设备未显示
blkid /dev/sdx
# (无输出)

# 原因: 设备没有文件系统或未分区

# 解决: 先创建文件系统
mkfs.xfs /dev/sdx
blkid /dev/sdx
# /dev/sdx: UUID="..." TYPE="xfs"

# 问题2: 找不到 UUID
blkid -s UUID -o value /dev/sda1
# (无输出)

# 原因: 标签不存在或设备未格式化

# 解决: 检查设备状态
lsblk -f /dev/sda1
# 如果 FSTYPE 为空, 需要先格式化
```

演示：文件系统实践

演示目标

掌握文件系统查看和挂载操作

演示环境要求

- RHEL 9 系统
 - 可选：额外的块设备（用于挂载演示）
 - 无需额外权限（大部分操作普通用户可执行，挂载需要 root）
-

演示步骤

步骤1：查看块设备结构

查看所有块设备和挂载情况

```
lsblk
```

预期输出示例:

```
# NAME      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
# vda       253:0    0   40G  0 disk
# └─vda1    253:1    0  200M  0 part /boot/efi
# └─vda2    253:2    0  500M  0 part /boot
# └─vda3    253:3    0 39.3G  0 part /
# vdb       253:16   0   10G  0 disk          <-- 未使用的额外磁盘
# └─vdb1    253:17   0    5G  0 part          <-- 未挂载的分区
```

查看带文件系统类型的详细信息

```
lsblk -f
```

预期输出示例:

```
# NAME FSTYPE LABEL UUID
MOUNTPOINT
# vda
# └─vda1 vfat    7B77-95E7          /boot/efi
# └─vda2 xfs     2d67e6d0-1f09-... /boot
# └─vda3 xfs     efd314d0-b56e-... /
# vdb
# └─vdb1 xfs     a1b2c3d4-5e6f-... <-- 有文件系统
但未挂载
```

说明:

- 查找 **MOUNTPOINT** 为空的设备 (表示未挂载)
- 如果 **FSTYPE** 为空, 说明设备没有文件系统, 需要先格式化

步骤2: 查看磁盘使用情况

查看所有文件系统的磁盘使用

```
df -h
```

预期输出示例:

```
# Filesystem      Size  Used Avail Use% Mounted on
# devtmpfs        892M   0  892M   0% /dev
# /dev/vda3       8.0G  1.4G  6.7G  17% /
# /dev/vda1       511M  101M  411M  20% /boot
# /dev/vda2       500M  250M  251M  50% /boot/efi
```

查看特定类型的文件系统

```
df -h -t xfs
```

预期输出 (只显示 XFS 文件系统) :

```
# Filesystem      Size  Used Avail Use% Mounted on
# /dev/vda3       8.0G  1.4G  6.7G  17% /
# /dev/vda2       500M  250M  251M  50% /boot
```

参数说明:

- **-h** = 人类可读格式
- **-t** = 指定文件系统类型
- **-x** = 排除指定类型

步骤3: 查看目录使用情况

```
# 查看特定目录的大小
du -sh /var/log

# 预期输出示例:
# 150M    /var/log

# 查看各个子目录的大小
du -sh /var/log/*

# 预期输出示例:
# 45M      /var/log/anaconda
# 2.5M     /var/log/audit
# 89M      /var/log/journal
# ...

# 查看最大的前5个目录
du -sh /var/* | sort -hr | head -5

# 预期输出示例:
# 1.2G     /var/lib
# 450M     /var/cache
# 150M     /var/log
# 50M      /var/spool
# 10M      /var/run
```

参数说明:

- `-s` = 只显示总计
- `-h` = 人类可读格式
- `sort -hr` = 按数值逆序排序

步骤4：挂载文件系统演示

注意：以下演示假设系统中有一个未挂载的分区 `/dev/vdb1`
如果没有额外设备，可以跳过此步骤，或使用 ISO 文件演示挂载

```
# 创建挂载点目录
mkdir -p /mnt/demo

# 确认挂载点为空目录
ls -la /mnt/demo

# 预期输出:
# total 0
# drwxr-xr-x. 2 root root 6 Jan 15 10:00 .
# drwxr-xr-x. 3 root root 20 Jan 15 10:00 ..

# 挂载文件系统
mount /dev/vdb1 /mnt/demo

# 验证挂载成功
df -h | grep demo

# 预期输出:
# /dev/vdb1          5.0G   50M   5.0G   1% /mnt/demo

# 查看挂载点内容
ls -la /mnt/demo

# 预期输出 (显示文件系统的内容) :
# total 20
# drwxr-xr-x. 2 root root 4096 Jan 15 09:00 .
# drwxr-xr-x. 3 root root   20 Jan 15 10:00 ..
# -rw-r--r--. 1 root root   10 Jan 15 09:00 test.txt
```

挂载方式说明：

方式	命令示例	说明
设备名	<code>mount /dev/vdb1 /mnt/demo</code>	简单，但不推荐（设备名可能变化）
UUID	<code>mount UUID="xxx" /mnt/demo</code>	推荐，UUID 稳定唯一
标签	<code>mount LABEL="mydisk" /mnt/demo</code>	使用卷标（需预先设置）
指定类型	<code>mount -t xfs /dev/vdb1 /mnt/demo</code>	明确指定文件系统类型

步骤5：卸载文件系统演示

SHELL

```
# 确认当前在挂载点外的目录
cd /root

# 方式1：使用挂载点卸载
umount /mnt/demo

# 或方式2：使用设备名卸载
# umount /dev/vdb1

# 验证卸载成功
df -h | grep demo
# 预期输出：空（说明已卸载）

# 查看挂载点内容（现在是空目录）
ls -la /mnt/demo
# 预期输出：
# total 0
```

步骤6：处理设备忙错误（演示）

```
# 重新挂载
mount /dev/vdb1 /mnt/demo

# 进入挂载点目录（模拟占用）
cd /mnt/demo

# 尝试卸载（会失败）
umount /mnt/demo

# 预期错误输出：
# umount: /mnt/demo: target is busy.

# 查看谁在使用该目录
lsof /mnt/demo

# 预期输出：
# COMMAND    PID USER   FD TYPE DEVICE SIZE/OFF NODE NAME
# bash       1234 root    cwd DIR  253,17    128   /mnt/demo
# 说明：bash 进程（当前 shell）正占用该目录

# 解决方案：退出该目录
cd /root

# 再次尝试卸载（成功）
umount /mnt/demo
```

演示总结

任务	命令	说明
查看块设备	lsblk	显示所有块设备和挂载点
查看文件系统类型	lsblk -f	显示文件系统类型和 UUID
查看磁盘使用	df -h	显示已挂载文件系统的使用情况
查看目录大小	du -sh 目录	显示目录占用的空间
创建挂载点	mkdir /mnt/xxx	创建空目录作为挂载点
挂载文件系统	mount 设备 挂载点	挂载文件系统到指定目录
卸载文件系统	umount 挂载点	卸载文件系统
查找占用进程	lsof 目录	查找占用目录的进程

故障排除

问题1：设备没有文件系统

SHELL

```
# 症状：
mount /dev/vdb1 /mnt/demo
# mount: /mnt/demo: unknown filesystem type 'xfs'

# 原因：设备未格式化

# 解决：先创建文件系统
mkfs.xfs /dev/vdb1
# 然后再挂载
mount /dev/vdb1 /mnt/demo
```

问题2：设备忙无法卸载

SHELL

```
# 症状：
umount /mnt/demo
# umount: /mnt/demo: target is busy.

# 解决1：查找并终止占用进程
lsof /mnt/demo
kill -9 <PID>

# 解决2：强制卸载（不推荐，可能导致数据丢失）
umount -f /mnt/demo

# 解决3：延迟卸载（断开连接后卸载）
umount -l /mnt/demo
```

问题3：挂载点目录不为空

```
# 症状:
mkdir /tmp/demo
echo "test" > /tmp/demo/file.txt
mount /dev/vdb1 /tmp/demo
# 挂载成功, 但原有文件被隐藏

# 说明: 挂载后, 原有目录内容被隐藏, 不会删除
# 卸载后会重新显示

# 建议: 使用空目录作为挂载点
```

练习名称	对应章节	说明
选择练习: IDENTIFYING FILE SYSTEMS AND DEVICES	第13章 第01节	识别文件系统和设备
练习: MOUNTING AND UNMOUNTING FILE SYSTEMS	第13章 第02节	挂载和卸载文件系统

第七部分：搜索文件

7.1 locate - 快速搜索

locate 命令：

```
# 搜索文件名
locate passwd

# 输出示例:
# /etc/passwd
# /etc/passwd-
# /etc/pam.d/passwd
# /usr/bin/passwd
```

特点：

- 搜索预生成的数据库

- 速度快
 - 数据库不是实时更新
-

updatedb - 更新数据库:

```
# 更新 locate 数据库
updatedb

# 说明:
# 数据库每日自动更新
# 手动更新确保结果准确
```

SHELL

locate 常用选项:

```
# 不区分大小写搜索
locate -i messages

# 限制结果数量
locate -n 5 passwd

# 输出示例:
# /etc/passwd
# /etc/passwd-
# /etc/pam.d/passwd
# /usr/bin/gpasswd
# /usr/bin/passwd
```

SHELL

参数说明:

- **-i** = ignore case 忽略大小写
 - **-n** = limit 限制结果数量
-

7.2 find - 实时搜索

find 命令基本语法:

`find [路径] [选项] [测试]`

⚠ 权限注意事项:

- `find` 命令在 classroom 环境中会产生大量 "Permission denied" 警告
- 这是正常现象, 某些系统目录需要 root 权限访问
- 使用 `2>/dev/null` 抑制错误输出, 只显示匹配结果
- 示例: `find /etc -name "*.conf" 2>/dev/null`

按文件名搜索:

```
# 按文件名搜索
find / -name sshd_config

# 输出示例:
# /etc/ssh/sshd_config

# 使用通配符搜索
find /etc -name "*pass*"

# 输出示例:
# /etc/security/opasswd
# /etc/pam.d/passwd
# /etc/passwd
```

参数说明:

- `-name` = 按名称搜索 (区分大小写)
- `-iname` = 按名称搜索 (不区分大小写)

按文件类型搜索:

```
# 搜索目录
find /etc -type d

# 搜索普通文件
find /etc -type f

# 搜索符号链接
find / -type l

# 搜索块设备
find /dev -type b
```

文件类型：

- **f** = 普通文件
- **d** = 目录
- **l** = 符号链接
- **b** = 块设备
- **c** = 字符设备

按文件大小搜索：

```
# 搜索大于10M的文件
find / -size +10M

# 搜索小于10K的文件
find / -size -10K

# 搜索正好10M的文件
find / -size 10M
```

大小单位：

- **b** = 512字节块
- **c** = 字节
- **w** = 双字节
- **k** = KB
- **M** = MB

- `G` = GB
-

按修改时间搜索：

SHELL

```
# 搜索120分钟前修改的文件
find / -mmin +120

# 搜索最近150分钟内修改的文件
find / -mmin -150

# 搜索正好120分钟前修改的文件
find / -mmin 120
```

参数说明：

- `-mmin` = 按修改分钟数
 - `+n` = 大于n
 - `-n` = 小于n
 - `n` = 等于n
-

按所有者搜索：

SHELL

```
# 搜索属于 root 的文件
find / -user root

# 搜索属于 student 用户的文件
find /home -user student

# 搜索属于 mail 组的文件
find /var -group mail
```

参数说明：

- `-user` = 按用户名
- `-group` = 按组名
- `-uid` = 按用户ID
- `-gid` = 按组ID

按权限搜索：

SHELL

搜索具有 SUID 权限的文件

```
find / -perm -4000
```

搜索具有 SGID 权限的文件

```
find / -perm -2000
```

搜索其他人有写权限的文件

```
find /home -perm -002
```

权限符号：

- `-perm mode` = 精确匹配权限
- `-perm -mode` = 权限位全部匹配
- `-perm /mode` = 权限位任一匹配

组合条件：

SHELL

搜索 /usr 下小于10M且具有SGID权限的文件

```
find /usr -size -10M -perm -2000
```

搜索属于 root 用户且是普通文件的文件

```
find / -user root -type f
```

使用 AND (默认)、OR、NOT

```
find / -name "*.txt" -o -name "*.pdf"
```

搜索非 .txt 文件

```
find / -name "*.txt" -not -name "*.log"
```

逻辑运算符：

- `-a` 或无 = AND
 - `-o` = OR
 - `-not` 或 `!` = NOT
-

find 执行命令：

SHELL

```
# 搜索并删除文件
find /tmp -name "*.tmp" -delete

# 搜索并复制文件
find / -user jacques -exec cp -a {} /root/findfiles \;

# 参数说明：
# -exec command \; = 对每个结果执行命令
# {} = 代表找到的文件
# \; = 命令结束符
```

⚠ 重点：find 高级用法

SHELL

```
# 常见使用场景汇总：

# 场景1：查找某个用户拥有的所有文件
find /home -user username -type f

# 场景2：查找大于100M的文件
find / -size +100M -type f

# 场景3：查找修改时间在7天内的文件
find /var/log -mtime -7 -type f

# 场景4：查找所有SUID文件（安全检查）
find / -perm -4000 -type f

# 场景5：查找并删除特定文件
find /tmp -name "*.swp" -delete

# 场景6：查找空文件
find /tmp -type f -empty

# 场景7：查找空目录
find /tmp -type d -empty

# 场景8：查找深度限制（只搜索前2层）
find /etc -maxdepth 2 -name "*.conf"

# 场景9：查找最近修改的文件
find /etc -name "*.conf" -mtime -30 -ls
```

7.3 find 时间参数详解

find 时间参数说明：

```
# 时间参数详解：  
# -mtime    = 修改时间（内容修改）  
# -atime    = 访问时间（文件被读取）  
# -ctime    = 状态改变时间（元数据修改）  
# -mmin     = 按分钟计算的修改时间  
# -amin     = 按分钟计算的访问时间  
# -cmin     = 按分钟计算的状态改变时间
```

SHELL

时间参数使用规则：

```
# n = 正好n  
# -n = 小于n（最近n单位内）  
# +n = 大于n（超过n单位前）  
  
# 示例：  
find /var/log -mtime 7      # 正好7天前修改  
find /var/log -mtime -7    # 7天内修改（最近7天）  
find /var/log -mtime +7    # 超过7天前修改（7天前）
```

SHELL

⚠ 易错点：时间参数

常见错误理解:

X 错误: `-mtime 7` 表示"最近7天"

正确: `-mtime 7` 表示"正好7天前"

X 错误: `-mtime -7` 表示"7天前"

正确: `-mtime -7` 表示"最近7天内"

记忆技巧:

- (减号) = 少于 = 最近

+ (加号) = 多于 = 以前

实际场景:

查找30天内修改的配置文件

```
find /etc -name "*.conf" -mtime -30
```

查找90天前未访问的文件

```
find /tmp -type f -atime +90
```

7.4 find 输出格式化

控制 **find** 输出格式:


```
# 显示详细信息 (类似 ls -l)
find /etc -name "*.conf" -ls

# 输出示例:
# 167897  4 -rw-r--r--    1 root root   217 1月  15 10:00 /etc/hosts
# 167898  4 -rw-r--r--    1 root root   512 1月  15 10:00
/etc/resolv.conf

# 只显示文件名 (默认)
find /etc -name "*.conf"

# 显示文件相对路径
find /etc -name "*.conf" -printf "%f\n"

# 格式化输出
find /etc -name "*.conf" -printf "%p %s bytes\n"

# 输出示例:
# /etc/hosts 217 bytes
# /etc/resolv.conf 512 bytes
```

printf 格式说明符:

格式符	说明	示例
<code>%p</code>	文件完整路径	/etc/hosts
<code>%f</code>	文件名 (不含路径)	hosts
<code>%s</code>	文件大小 (字节)	217
<code>%u</code>	所有者用户ID	0
<code>%g</code>	所有者组ID	0
<code>%M</code>	权限 (rwxrwxrwx)	rw-r--r--
<code>%t</code>	修改时间 (时间戳)	1642252800
<code>\n</code>	换行符	-

⚠ 技巧: **find** 的 **-printf**

```
# 实际场景：列出所有 .conf 文件及其大小
find /etc -name "*.conf" -printf "%p %s bytes\n"

# 对比 -ls 选项：
find /etc -name "*.conf" -ls
# -ls 输出类似 ls -l, 格式固定

# -printf 可以自定义格式
# 实际工作中可能要求特定输出格式
```

7.5 find 综合练习

练习1：查找用户文件

```
# 场景：
# 查找用户 jacques 拥有的所有文件，并复制到 /root/findfiles 目录

# 答案：
find / -user jacques -type f -exec cp -a {} /root/findfiles \;

# 说明：
# -user jacques    = 属于 jacques 用户
# -type f          = 只查找普通文件
# -exec cp -a      = 复制（保留属性）
# {}              = 代表找到的文件
# \;              = 命令结束符

# 验证：
ls -l /root/findfiles
```

练习2：查找大文件

```
# 场景：
# 查找 /var 目录下大于 10M 的普通文件

# 答案：
find /var -type f -size +10M

# 输出文件大小：
find /var -type f -size +10M -exec ls -lh {} \;

# 或使用 -ls：
find /var -type f -size +10M -ls
```

练习3：查找特定权限文件

```
# 场景：
# 查找 /home 目录下所有其他人有写权限的文件

# 答案：
find /home -perm -002 -type f

# 说明：
# -perm -002      = 其他用户有写权限
# -type f         = 只查找普通文件

# 权限位对照：
# 4000 = SUID    (set-user-id)
# 2000 = SGID    (set-group-id)
# 1000 = Sticky Bit
# 0400 = 用户读
# 0200 = 用户写
# 0100 = 用户执行
# 0040 = 组读
# 0020 = 组写
# 0010 = 组执行
# 0004 = 其他读
# 0002 = 其他写
# 0001 = 其他执行
```

练习4：按时间查找

```
# 场景：
# 查找 /etc 目录下最近 30 天内修改过的 .conf 文件

# 答案：
find /etc -name "*.conf" -mtime -30 -type f

# 按大小排序输出：
find /etc -name "*.conf" -mtime -30 -type f -exec ls -lh {} \;
```

演示：文件搜索实践

演示目标

掌握 find 和 locate 搜索文件

演示环境要求

- RHEL 9 系统
 - 无需特殊权限（大部分操作普通用户可执行）
 - 建议在限制目录内演示（如 `/etc`），避免搜索整个系统
-

演示步骤

步骤1：使用 **locate** 快速搜索

```
# 搜索包含 "passwd" 的文件名
locate passwd

# 预期输出示例:
# /etc/passwd
# /etc/passwd-
# /etc/pam.d/passwd
# /usr/bin/passwd
# /usr/share/man/man1/passwd.1.gz
# /usr/share/man/man5/passwd.5.gz
# ...

# 不区分大小写搜索 (查找包含 "ssh" 的文件)
locate -i ssh

# 预期输出示例 (包含 SSH、ssh 等大小写组合) :
# /etc/ssh
# /etc/ssh/sshd_config
# /usr/bin/ssh
# /usr/sbin/sshd
# ...

# 限制结果数量为 10 个
locate -n 10 passwd

# 预期输出 (只显示前10个结果) :
# /etc/passwd
# /etc/passwd-
# /etc/pam.d/passwd
# /usr/bin/passwd
# /usr/share/bash-completion/completions/passwd
# /usr/share/man/man1/chpasswd.1.gz
# /usr/share/man/man1/passwd.1.gz
# /usr/share/man/man5/passwd.5.gz
# /usr/share/man/man8/chpasswd.8.gz
# /usr/share/man/man8/vipw.8.gz
```

说明:

- `locate` 搜索预建的数据库, 速度非常快
- 数据库每日自动更新
- 新创建的文件可能搜索不到, 需要 `updatedb` 更新

步骤2：使用 **find** 按名称精确搜索

SHELL

```
# 在 /etc 中搜索包含 "pass" 的文件 (区分大小写)
find /etc -name "*pass*"

# 预期输出示例:
# /etc/passwd
# /etc/passwd-
# /etc/pam.d/passwd
# /etc/security/opasswd

# 在整个系统中搜索 sshd_config (精确文件名)
find / -name sshd_config 2>/dev/null

# 预期输出:
# /etc/ssh/sshd_config

# 不区分大小写搜索 (查找包含 "network" 的文件)
find /etc -iname "*network*"

# 预期输出示例 (包含 NETWORK、network、Network 等) :
# /etc/sysconfig/network
# /etc/sysconfig/network-scripts
# /etc/NetworkManager
# ...
```

参数说明：

- **-name** = 区分大小写
- **-iname** = 不区分大小写
- ***** = 通配符，匹配任意字符
- **2>/dev/null** = 隐藏权限错误输出

步骤3：使用 **find** 按文件类型搜索

搜索 /etc 下的所有目录

```
find /etc -type d | head -10
```

预期输出示例:

```
# /etc
# /etc/NetworkManager
# /etc/NetworkManager/dispatcher.d
# /etc/NetworkManager/dnsmasq-shared.d
# /etc/alsa
# /etc/audit
# /etc/audit/rules.d
# ...
```

搜索 /etc 下的所有普通文件

```
find /etc -type f | head -10
```

预期输出示例:

```
# /etc/fstab
# /etc/crypttab
# /etc/mtab
# /etc/passwd
# /etc/group
# ...
```

搜索 /dev 下的块设备文件

```
find /dev -type b | head -10
```

预期输出示例:

```
# /dev/sda
# /dev/sda1
# /dev/sda2
# /dev/vda
# /dev/vda1
# ...
```

搜索 /dev 下的字符设备文件

```
find /dev -type c | head -10
```

预期输出示例:

```
# /dev/null
# /dev/zero
# /dev/full
# /dev/random
```

```
# /dev/urandom
# ...
```

文件类型说明:

类型	说明	示例
-----	-----	-----
f	普通文件	/etc/passwd
d	目录	/etc
l	符号链接	/usr/bin/sh → bash
b	块设备	/dev/sda
c	字符设备	/dev/null

步骤4: 使用 **find** 按文件大小搜索

SHELL

```
# 搜索 /var 下大于 10M 的文件
find /var -size +10M -exec ls -lh {} \;

# 预期输出示例:
# -rw-----. 1 root root 150M Jan 15 09:00
/var/log/journal/xxx/system.journal
# -rw-r--r--. 1 root root 25M Jan 14 10:00 /var/lib/rpm/Packages

# 搜索 /tmp 下小于 1K 的文件
find /tmp -size -1K -ls

# 预期输出示例:
# 16777230 0 -rw-rw-r-- 1 root root 0 Jan 15 10:00
/tmp/test.txt

# 搜索正好 4K 的文件
find /etc -size 4k

# 预期输出示例 (显示恰好 4K 的文件) :
# /etc/adjtime
# /etc/environment
```

大小单位说明:

单位	说明
-----	-----

b	512 字节块
c	字节
w	双字节 (2 字节)
k	KB (1024 字节)
M	MB
G	GB

符号说明:

- **+10M** = 大于 10M
- **-10M** = 小于 10M
- **10M** = 等于 10M

步骤5: 使用 **find** 按修改时间搜索

SHELL

```
# 搜索 /etc 下最近 7 天内修改过的 .conf 文件
find /etc -name "*.conf" -mtime -7
```

```
# 预期输出示例:
```

```
# /etc/ssh/sshd_config
```

```
# /etc/resolv.conf
```

```
# 搜索 /var/log 下超过 30 天前的日志文件
```

```
find /var/log -name "*.log" -mtime +30
```

```
# 预期输出示例 (显示旧日志) :
```

```
# /var/log/old-2024-01.log
```

```
# /var/log/archive.log
```

```
# 搜索最近 120 分钟内修改的文件
```

```
find /etc -mmin -120
```

```
# 预期输出示例:
```

```
# /etc/resolv.conf
```

```
# /etc/hosts
```

时间参数说明:

参数	单位	说明
-----	-----	-----
-mtime	天	修改时间 (内容)

	-atime		天		访问时间（读取）	
	-ctime		天		状态改变时间（元数据）	
	-mmin		分钟		修改时间（分钟）	
	-amin		分钟		访问时间（分钟）	

⚠ 重要：时间参数符号

- -7 = 少于 7（最近 7 天内）
- +7 = 大于 7（超过 7 天前）
- 7 = 正好 7

步骤6：使用 **find** 组合条件搜索

```
# 搜索 /etc 下以 .conf 结尾且大于 10K 的文件
find /etc -name "*.conf" -size +10k

# 预期输出示例:
# /etc/selinux/targeted/policy/policy.32

# 搜索属于 root 用户且是普通文件的文件
find /etc -user root -type f | head -10

# 预期输出示例:
# /etc/passwd
# /etc/group
# /etc/fstab
# ...

# 搜索 .txt 或 .pdf 文件 (OR 条件)
find /home -name "*.txt" -o -name "*.pdf"

# 预期输出示例:
# /home/user/document.txt
# /home/user/manual.pdf

# 搜索不是 .log 的文件 (NOT 条件)
find /var/log -type f ! -name "*.log" | head -10

# 预期输出示例:
# /var/log/wtmp
# /var/log/btmp
# /var/log/lastlog
```

逻辑运算符:

- 默认 (或 **-a**) = AND, 同时满足
- **-o** = OR, 满足任一
- **!** 或 **-not** = NOT, 不满足

步骤7: 使用 find 执行命令

```
# 搜索并显示文件的详细信息
find /etc -name "*.conf" -ls

# 预期输出示例:
# 167897  4 -rw-r--r--    1 root root   217 Jan 15 10:00 /etc/hosts
# 167898  4 -rw-r--r--    1 root root   512 Jan 15 10:00
/etc/resolv.conf

# 搜索并列出文件权限和大小
find /etc -name "*.conf" -exec ls -lh {} \;

# 预期输出示例:
# -rw-r--r--. 1 root root   217 Jan 15 10:00 /etc/hosts
# -rw-r--r--. 1 root root   512 Jan 15 10:00 /etc/resolv.conf

# 查找所有 .conf 文件并统计数量
find /etc -name "*.conf" | wc -l

# 预期输出:
# 45
```

参数说明:

- `-exec command \;` = 对每个结果执行命令
- `{}` = 代表找到的文件
- `\;` = 命令结束符 (必须)

演示总结

工具	优点	缺点	适用场景
locate	极快	数据库可能过时	快速定位已知文件
find	实时准确	搜索较慢	精确查找、复杂条件

find 常用选项速查:

选项	说明	示例
-name	按名称 (区分大小写)	find /etc -name "hosts"
-iname	按名称 (不区分大小写)	find /etc -iname "HOSTS"

选项	说明	示例
<code>-type</code>	按文件类型	<code>find /etc -type f</code>
<code>-size</code>	按大小	<code>find /var -size +10M</code>
<code>-mtime</code>	按修改天数	<code>find /etc -mtime -7</code>
<code>-user</code>	按所有者	<code>find /home -user student</code>
<code>-perm</code>	按权限	<code>find / -perm -4000</code>
<code>-exec</code>	执行命令	<code>find /etc -exec ls -l {} \;</code>

今日总结

命令清单

软件包查询:

```
rpm -qa           # 列出所有已安装软件包
rpm -q package    # 查看软件包版本
rpm -ql package   # 列出软件包文件
rpm -qf file       # 查找文件所属软件包
```

SHELL

软件包管理:

```
dnf list          # 列出软件包
dnf install package # 安装软件包
dnf remove package # 删除软件包
dnf search keyword # 搜索软件包
dnf repolist       # 列出仓库
```

SHELL

文件系统:

```
lsblk            # 列出块设备
lsblk -f         # 查看文件系统信息
df -h           # 查看磁盘使用
du -sh directory # 查看目录大小
```

SHELL

文件搜索:

SHELL

```
locate keyword          # 快速搜索文件
find /path -name pattern # 按名称搜索
find /path -type d       # 搜索目录
find /path -size +10M    # 按大小搜索
```

常见任务

1. 查找文件属于哪个软件包

SHELL

```
rpm -qf /usr/bin/ls
```

2. 查找某个用户拥有的文件

SHELL

```
find / -user username
```

3. 查找特定大小的文件

SHELL

```
find /path -size +10M
```

4. 配置本地yum仓库

SHELL

```
vim /etc/yum.repos.d/local.repo
# [baseos]
# name=BaseOS
# baseurl=http://...
# enabled=1
# gpgcheck=0
```

5. 列出软件包安装的文件

SHELL

```
rpm -ql package-name
```

练习名称	对应章节	说明
练习： LOCATING FILES ON THE SYSTEM	第13章 第03节	在系统上定位文件
总练习： ACCESSING LINUX FILE SYSTEMS	第13章	访问Linux文件系统综合练习